



CESDE

Santiago Yosa

Ingeniero Sistemas de Información



Unidad II

JPA+HIBERNATE CONEXIÓN CON BD

- Configuración conexión con BD
- Mapeo de entidades
- Relaciones entre entidades
- Operaciones de persistencia CRUD

SISTEMA CONTROL DE VERSIONES

- Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Paso 0: Puente Cognitivo

Concepto a dominar:

El uso de `@Entity` y la herencia con `@MappedSuperclass`. Debemos entender que una entidad es una representación de una fila en una tabla.

Reto rápido

Tienes la clase `Avion` y la clase `Piloto`. Ambos son entidades con su propio ID.

Pregunta:

¿Cómo harías en SQL para decirle a un `Piloto` qué `Avion` tiene asignado?

Situación

En nuestra aplicación de inventario, un Producto raramente existe en el vacío. Un "Teclado Mecánico" pertenece a la categoría de "Tecnología"; una "Libreta de Tapa Dura" a la de "Papelería". Si tuviéramos 100 productos de tecnología, sería ineficiente y propenso a errores escribir la palabra "Tecnología" en cada uno de ellos. ¿Qué pasa si un día queremos cambiar el nombre de la categoría a "Electrónica"? Tendríamos que actualizar 100 registros manualmente.

En el mundo de las bases de datos, esto se resuelve con una tabla categorías y una **clave foránea** en la tabla productos. En el mundo de la Programación Orientada a Objetos, esto se resuelve con un objeto Producto que contiene una referencia a un objeto Categoría.

El desafío es: ¿cómo le enseñamos a JPA/Hibernate a traducir esta relación de objetos en una estructura de tablas y claves foráneas eficiente?

Relaciones entre Entidades

JPA nos permite modelar las relaciones de bases de datos directamente con anotaciones. La más común es la relación "muchos a uno".

Conceptos Clave de Relaciones:

- **Cardinalidad:** Describe la relación entre dos tablas:
 - **Muchos a Uno (@ManyToOne):** Muchos Productos pueden pertenecer a una Categoría.
 - **Uno a Muchos (@OneToMany):** Una Categoría puede tener muchos Productos.
 - **Uno a Uno (@OneToOne):** Un Usuario tiene un PerfilDeUsuario.
 - **Muchos a Muchos (@ManyToMany):** Muchos Estudiantes pueden estar en muchos Cursos.
- **Direccionalidad:**
 - **Unidireccional:** Solo una entidad "conoce" la relación. Por ejemplo, Producto conoce a Categoría, pero Categoría no sabe qué productos tiene.
 - **Bidireccional:** Ambas entidades se conocen mutuamente. Producto conoce a su Categoría, y Categoría tiene una lista de sus Productos.

Lado Propietario (Owning Side): En una relación, una de las dos entidades es la "propietaria". Es la que tiene la columna de clave foránea en la base de datos. **Por regla general, el lado @ManyToOne es siempre el propietario.**

Anatomía del Código: @ManyToOne y @OneToMany

En la entidad Producto (el lado "Muchos"):

- **@ManyToOne**: Define la cardinalidad.
- **@JoinColumn**: Es crucial. Le dice a Hibernate: "Crea una columna en la tabla productos llamada categoria_id que almacenará el ID de la categoría correspondiente".

En la entidad Categoria (el lado "Uno"):

- **@OneToMany**: Define la cardinalidad inversa.
- **mappedBy = "categoria"**: Esto es fundamental. Le dice a Hibernate: "No crees una clave foránea aquí. La gestión de esta relación ya la hace el campo categoria en la clase Producto". Sin esto, Hibernate crearía una tabla intermedia innecesaria.
- **cascade = CascadeType.ALL**: Propaga las operaciones de persistencia. Si guardas una categoría, también se guardarán los productos de su lista. Si la borras, se borrarán sus productos.
- **orphanRemoval = true**: Si quitas un producto de la lista de una categoría y guardas la categoría, el producto "huérfano" será eliminado de la base de datos.

Relación Unidireccional

@Entity

```
public class Estudiante extends BaseEntity { // BaseEntity tiene el ID (MappedSuperclass)
    private String nombre;
```

```
    // Relación: Muchos estudiantes -> Un curso
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "id_curso") // Define el nombre de la columna FK en la tabla
```

```
    private Curso curso;
```

```
    public Estudiante() {}
```

```
    // Getters y Setters...
```

```
}
```

Preguntas Frecuentes

Pregunta: ¿Qué pasa si borro un Curso que tiene Estudiantes?

Por defecto, la BD lanzará un error de "Integridad Referencial". JPA permite configurar el cascade, pero por seguridad, no se debe borrar un padre si tiene hijos asociados sin gestionar esa relación primero.

Pregunta: ¿Cuándo debería usar una relación unidireccional en lugar de una bidireccional?

Usa la simplicidad como guía. Si en tu lógica de negocio **nunca** necesitas navegar desde una Categoría para obtener su lista de Productos, entonces una relación unidireccional (solo con @ManyToOne en Producto) es suficiente y más simple. Hazla bidireccional solo cuando realmente necesites acceder a la relación desde ambos lados.

Relaciones entre Entidades

Así es como mapeamos las claves foráneas del mundo SQL al mundo de los objetos.

- **@ManyToOne:** La relación más común. Se pone en la entidad "hija". Por ejemplo, muchos Productos pertenecen a una Categoría. La clase Producto tendría un campo Categoría anotado con @ManyToOne. Esto crea la columna de clave foránea en la tabla de productos.
- **@OneToMany:** Es la otra cara de la moneda. Se pone en la entidad "padre". La Categoría tendría una List<Producto> anotada con @OneToMany. El atributo mappedBy es crucial aquí para decirle a Hibernate que la relación ya está gestionada por el lado @ManyToOne, evitando la creación de tablas o columnas extra.
- **Cascading (cascade = CascadeType.ALL):** Una opción muy útil que le dice a Hibernate: "Si guardo una Categoría, guarda también todos los Productos en su lista. Si la borro, borra también sus productos". Simplifica las operaciones.

Mapecto de Relaciones

@OneToMany (Una Categoría a Muchos Productos): Una Categoría tiene muchos Productos. Se coloca en la clase Categoría sobre la lista de productos.

Concepto: Una instancia de esta entidad se relaciona con muchas instancias de otra.

Ejemplo: Un Categoría puede tener muchas Productos.

Implementación:

```
// En la clase Categoría (lado inverso)
```

```
@OneToMany(mappedBy = "Categoría") // "Categoría" es el nombre del atributo en la clase Productos
```

```
private List< Productos> productos;
```

@ManyToOne (Muchos Productos a Una Categoría): Muchos Productos pertenecen a una única Categoría. Se coloca en la clase Producto sobre el atributo de la categoría.

Concepto: Muchas instancias de esta entidad están relacionadas con una instancia de otra entidad.

Ejemplo: Muchos Productos pertenecen a un Categoría. El propietario es Productos, porque la tabla Productos contendrá la llave foránea Categoría_id.

Implementación:

```
// En la clase Producto (lado propietario)
```

```
@ManyToOne
```

```
@JoinColumn(name = "Categoría_id") // Define la columna de la llave foránea
```

```
private Categoría categoría;
```

`spring.jpa.hibernate.ddl-auto:`

Es una de las propiedades más importantes. Controla cómo Hibernate interactúa con el esquema de la base de datos.

- **none:** No hace nada (la opción más segura en producción).
- **update:** Actualiza el esquema de la base de datos para que coincida con las entidades de tu código.
- **create:** Borra todas las tablas y las crea de nuevo.
- **create-drop:** Hace lo mismo que create pero borra las tablas al cerrar la aplicación.

Ejemplos demostrativos - Un sistema de Autores y Libros.

```
@Entity
public class Autor {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    // Constructores, Getters...
}
```

```
@Entity
public class Libro {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;

    @ManyToOne
    @JoinColumn(name = "autor_id")
    private Autor autor;
    // Constructores, Getters...
}
```

Lado Propietario (Owning Side): En una relación, una de las dos entidades es la "propietaria". Es la que tiene la columna de clave foránea en la base de datos. **Por regla general, el lado @ManyToOne es siempre el propietario.**

Práctica acompañada - Learning by Doing

Ejemplo de aprendizaje (Departamento y Empleado)

Instrucciones:

1. Crea un proyecto en springboot y descarga las dependencias.
2. Crea la entidad Departamento (id, nombre).
3. Crea la entidad Empleado (id, nombre, cargo).
4. Establece una relación @ManyToOne de Empleado hacia Departamento.
5. Configura tu application properties
6. Ejecuta y verifica en la consola H2 (localhost:8080/h2-console) que la tabla empleado tiene una columna departamento_id.
7. Versiona lo aprendido en un nuevo repositorio.

Troubleshooting (Solución de Errores Comunes)

TransientPropertyValueException:

- **Error:** Intentas guardar un Libro pasando un Autor que aún no ha sido guardado en la base de datos.
- **Solución:** Primero guarda (save) al padre (Autor) y luego al hijo (Libro).

StackOverflowError (Recursión Infinita):

- **Error:** Ocurre en relaciones bidireccionales cuando imprimes un objeto con toString(). El Padre llama al Hijo, el Hijo al Padre... infinito.
- **Solución:** No incluyas el objeto relacionado en el método toString() o usa @JsonIgnore en niveles superiores.

La Súper-clase

No le ponemos @Entity porque no queremos una tabla llamada "persona".

```
import jakarta.persistence.*;
```

```
@MappedSuperclass
```

```
public abstract class Persona {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false)
```

```
    private String nombre;
```

```
    @Column(unique = true, nullable = false)
```

```
    private String correo;
```

```
    public Persona() {} // Constructor obligatorio de JPA
```

```
    // ... Getters y Setters
```

```
}
```

Las Entidades Independientes

Estos hijos heredan automáticamente id, nombre y correo de su padre.

```
@Entity
```

```
@Table(name = "medicos")
```

```
public class Medico extends Persona {
```

```
    private String especialidad;
```

```
    private String pabellones;
```

```
    public Medico() {}
```

```
    // ... Getters y Setters
```

```
}
```

```
@Entity
```

```
@Table(name = "enfermeros")
```

```
public class Enfermero extends Persona {
```

```
    private String responsabilidades;
```

```
    public Enfermero() {}
```

```
    // ... Getters y Setters
```

```
}
```

Un enfermero puede atender a **muchos** pacientes, por lo tanto, la relación desde el punto de vista del paciente es de **Muchos a Uno (@ManyToOne)**.

```
@Entity
```

```
@Table(name = "pacientes")
```

```
public class Paciente extends Persona {
```

```
    private String diagnostico;
```

```
    private String medicamentos;
```

```
    // EL ENFOQUE ORM: No guardamos el Long idEnfermero.
```

```
    // Guardamos la referencia al objeto Enfermero.
```

```
    @ManyToOne // Le damos un nombre claro a la Llave Foránea en SQL
```

```
    @JoinColumn(name = "enfermero_asignado_id")
```

```
    private Enfermero enfermeroAsignado;
```

```
    public Paciente() {}
```

```
    // ... Getters y Setters
```

```
}
```

Ejercicio independiente aplicado (Ticket Jira)

TICKET: JIRA-102 - Sistema Hospitalario

Descripción: Necesitamos registrar Pacientes. Cada paciente debe estar asignado obligatoriamente a un Médico de cabecera.

Requerimientos Técnicos:

1. Crear entidad Medico (id, especialidad).
2. Crear entidad Paciente (id, nombre).
3. Relacionar @ManyToOne Paciente -> Medico.
4. El nombre de la columna en la base de datos debe ser medico_asignado_id.
5. Configurar el application.properties para que las tablas se borren y creen cada vez (create-drop) para pruebas limpias.

Criterios de logro:

- El código compila sin errores.
- La tabla paciente muestra la columna con el nombre solicitado.
- Uso de Git con al menos 2 commits descriptivos.

¿Debería saber algo más?

Lazy vs Eager:

Por defecto, @ManyToOne es **Eager** (trae los datos del padre de inmediato). En sistemas grandes, esto puede ser lento. En el futuro aprenderemos a usar FetchType.LAZY.

Lombok:

Para evitar escribir Getters/Setters manualmente, los desarrolladores usan una librería llamada *Lombok*. Es recomendable aprenderla una vez domines la estructura manual de JPA.

CESDE | comfama